#75 大失業時代の中で最強の人材になれる。AI力が確実に伸びて育つ、ガチ勢のAI習慣

https://www.youtube.com/watch?v=sZNkoYbmRFk ／ 56:39

3行要約

- AIの進化が速すぎて、覚えるべき”当たり前”が100〜200個に増え、しかも「何が正しいか」を初心者が見分けにくい時代になった。
- 正しいやり方を見分ける目は、自分で触って壁にぶつかった人にしか育たない。
- 伸びる人の習慣は「うまくいかなかった理由を考え、昨日より少しでも良い使い方をする」こと。

主なポイント

- Astraのように人間の道具（画面）を理解するAIが出た以上、今は「プロンプトのコツが必要」でも、半年〜1年でコツ不要になっていく見込み。公開されたテクニックの優位性は3〜6か月程度で消える。
- 不要になった知識（コードの細かい修正など）もある一方、新しく必要な知識（コンテキスト・ハーネス・サブエージェント・スキル、データの学習オフ、メモリのオン/オフ等）が大量に増えた。
- ネットの解説記事には「現場の人が実際にどう使っているか」がほとんど書かれていない。例：スキルを作るときは「固有の値（変数）は外に出し、誰でも使える一般的な形にして」と頼む／スキルが劣化していないか確かめるテスト用データを用意する。
- 人によって正反対のことを言う（「スキルは丁寧に整備」vs「作らない方がいい」）のは、大規模開発か個人の文系業務かなど、背景が違うから。複数の流派をつまみ食いすると考え方の一貫性が崩れやすい。
- 「AIに詳しい」と名乗る講師や研修でも、古い情報のままのことが多い。見た目がきちんとしているかでは正しさは判断できない。
- 話題：米国でAI開発の速度を緩める動きがあるという話（真偽・実現性は不明）。もし本当なら、遅れている人が追いつくチャンス。
- 🔑 **成長が止まる人は、失敗しても同じ頼み方を繰り返す。「なぜできなかったか」「なぜ時間がかかったか」を考え、次は別のやり方を試す。**（例：

画面と中身を同時に直さず、いったん画面を捨てて中身を先に作り、最後に画面を作り直す）

- 🔑 **「できなかったこと・詰まったこと」を頭の中にストックしておく。課題意識があると、動画やSNSで解決策を見たときに「これだ」と気づける。** 課題意識がないと、良いコツも素通りしてしまう（→#74）。
- ミスのたびに設定ファイルへ「〇〇するな」と書き足すと、関係ない場面でそのルールが発動して逆効果になることがある。AIの装備（スキル・MCP）は軽くし、過去の失敗や判断材料は外付けの文脈として持ち、必要な分だけ読ませる（→#53）。
- 視聴者質問への回答：ローカルLLMの実用例は、外部サービスに出せない機密情報だけを扱う、個人単位の作業など限定的。会社全員で使うような用途には向かない（→#71）。Grok Botは「軽量なサブのAIエージェント」として、毎日のレポートやニュース収集に便利（→#72）。

紹介されたテクニック

- **スキルは「固有の値を外に出して一般化して」と頼む**：スキル（AIに覚えさせる作業手順書）を作るとき、「変数（その時だけの名前や数字など）は外に出して、誰でも使える一般的な形にして」と一言添える。これがないと、作ったスキルを次に使っても役に立たないことが多い。
- **スキルが劣化していないか確かめるテストデータを用意する**：AIのモデルが新しくなるとスキルの効き方が変わることがある。同じ入力で結果を比べられる確認用データを持っておき、ちゃんと動くかを点検する。
- **基本の設定を押さえる**：AIの学習にデータを使わせない設定（学習オフ）にしておく、会話が過去の記憶に引っ張られるならメモリ機能をオフにする、など「当たり前の設定」を確認しておく。
- **失敗したら同じ頼み方を繰り返さない**：「直して」を繰り返すだけでなく、なぜできなかったか・なぜ時間がかかったかを考え、次は別のやり方を試す。例えば「1週間かかった作業を1日でやるには？」と考えてみる。
- **画面と中身は分けて作る**：アプリの画面と裏側の仕組みを同時に直させると、間違いが増えトークン（AIの利用量）も食う。いったん画面を捨てて裏側の仕組みを先に作り、最後に画面を作り直す方が早いことがある。
- **見た目は「スクショを撮りながら直し続けて」と頼む**：「このデザインになるまでスクリーンショットを撮って確認し、修正を繰り返して」と指示すると、AIが自分で見比べて近づけていく。
- **「できなかったこと」を頭の中にストックする**：課題意識を持っていると、動画やSNSで解決策を見たときに「これだ」と気づける。その状態で過去の動画を見直すと、以前は素通りしたコツが見つかる。

- **設定ファイルに「〇〇するな」を書き足しすぎない**：ミスのたびにCLAUDE.md（Claude Codeが毎回読む設定ファイル）にルールを足すと、関係ない場面で発動して逆効果になる。AIの装備（スキル・MCP＝外部ツールとの接続）は軽くし、過去の失敗や判断材料は外付けの文脈として持って必要な分だけ読ませる。
- **Grok Botは定期作業のサブ役に**：毎日の売上レポート・ニュース収集・タスクリストなどをボットごとに分けて定期実行させると、PCをつけっぱなしにせず手軽に回せる。画面を使うPC操作をGrok側のPCに任せ、進捗を画像で送らせる使い方も。
- **ローカルLLMは機密部分だけに使う**：外部サービスに出せない情報（パスワード入りの作業、個人情報を伏せる処理など）だけをローカルLLM（自分のPC内で動くAI）に任せる。会社全員で使う用途には向かない。

紹介されたツール GPT-6 Astra（Codex）、Fable、Claude Code（CLAUDE.md）、Grok Bot、Obsidian、ローカルLLM、AWS Bedrock など

すぐ実践できること

- AIでうまくいかなかったこと・時間がかかったことを毎回メモし、「次はどう頼むか」を1行書き添える。
- 過去の動画や記事を、自分の課題を思い浮かべながら見直してみる（以前は気づかなかったコツが見つかる）。
- 設定ファイルにルールを書き足しすぎていないか見直し、不要なものは削る。